PROOFLINE

Agent Outcome Assurance Platform

Project Report

Problem, Architecture, End-to-End Workflow, Folder Structure and Phase-wise Delivery Plan

Version 1.0 | 24 October 2026

Table of Contents

1. Executive Summary **3**

2. Problem Statement and Motivation **4**

2.1 Problem statement . . . 4

2.2 Objectives . . . 4

3. Scope Definition **5**

3.1 Target users . . . 5

3.2 In scope and deferred . . . 5

3.3 Non-goals . . . 5

4. Advantages, Limitations and Trade-offs **6**

4.1 Advantages . . . 6

4.2 Limitations and mitigations . . . 6

5. Six Improvements That Strengthen the Project **7**

5.1 “Universal” does not mean automatic support for every model . . . 7

5.2 The AI judge must not be the final authority . . . 7

5.3 Trace and replay limitations must be explicit . . . 7

5.4 Action Guard must sit on the real execution boundary . . . 8

5.5 Each AI domain needs its own evaluation metrics . . . 8

5.6 The scope is too large to build at once . . . 8

5.7 Governing principle: modular monolith first . . . 8

5.8 Development priority . . . 9

5.9 Readiness verdict . . . 9

6. System Architecture **10**

6.1 Architectural principles . . . 10

6.2 Logical architecture . . . 10

6.3 Integration modes: visibility versus enforcement . . . 10

6.4 Component responsibilities . . . 11

6.5 Technology stack . . . 11

6.6 Evaluation architecture . . . 11

6.7 Action Guard architecture . . . 12

6.8 Trace and replay . . . 12

6.9 Domain-specific evaluators . . . 13

6.10 Core metrics . . . 13

6.11 Failure taxonomy . . . 13

6.12 Data model . . . 14

6.13 Interface contracts and error handling . . . 14

7. Repository and Folder Structure 15

8. Phase-wise Delivery Plan 18

Phase 0: Specification and Interface Contracts . . . 18

Phase 1: Foundation, Runtime and Trace . . . 19

Phase 2: Evaluation and the Claim-versus-State Verifier . . . 20

Phase 3: Action Guard and Tool Gateway . . . 21

Phase 4: Regression, Model Comparison, Replay and CI Gate . . . 22

Phase 5: Watchdog, Security Hardening and SDK . . . 23

Phase 6: Scout: Permission-gated Black-box Audit (Expansion) . . . 24

Phase 7: Deployment and Launch . . . 25

9. End-to-End Runtime Workflow 26

9.1 Failure-to-fix loop . . . 26

10. Testing and Quality Strategy 27

10.1 Proofline Bench . . . 27

10.2 Acceptance gates . . . 27

11. Deployment, Security and Operations 28

11.1 Environments . . . 28

11.2 CI/CD pipeline . . . 28

11.3 Security controls . . . 28

11.4 Runbook: principal incidents . . . 28

12. Risk Register and Cut Ladder 29

12.1 Cut ladder . . . 29

13. Conclusion 29

1. Executive Summary

Proofline is an **Agent Outcome Assurance** platform. Most AI evaluation tools answer the question “Was the answer good?” Proofline answers a harder and more valuable question: **“What did the AI agent actually do, what went wrong, what is the evidence, and did the fix make the system more reliable?”**

The platform executes an AI system under controlled conditions, records every step as a trace, verifies the agent’s claims against real tool results and database state, blocks unauthorized actions at the execution boundary, and compares each new version against a trusted baseline before it is released. Its central differentiator is the **claim-versus-state verifier**: when an agent says “your refund has been issued,” Proofline checks whether the refund exists in the system of record.

Key design decisions

- **Wedge, not a universe.** The architecture is model-agnostic, but the first product focuses on tool-using support and operations agents.
- **Modular monolith first.** One FastAPI application with strictly separated modules; microservices only when load and deployment needs justify them.
- **Three-layer evaluation.** Deterministic checks first, reference-based checks second, AI-based judgement last and always labelled.
- **Enforcement at the boundary.** Action Guard protects only what passes through a controlled runtime or Tool Gateway, and this limit is documented openly.
- **Phase-wise delivery.** Eight phases (0 to 7), each closed by an exit test rather than a calendar date.

Document purpose and reading guide

This report is the end-to-end engineering reference for Proofline. Sections 2–4 explain the problem, scope and the advantages and limitations. Section 5 lists the six improvements that make the design implementation-ready. Sections 6–7 define the architecture and repository layout. Section 8 gives the detailed phase-wise plan. Sections 9–12 cover the runtime workflow, testing, deployment and risks.

2. Problem Statement and Motivation

AI agents now take real actions: issuing refunds, cancelling orders, updating records and sending messages. Their failures are therefore no longer only “bad answers.” They are operational incidents with financial, legal and trust consequences.

Failure pattern	What happens	Why text-only evaluation misses it
False success claim	Agent says the refund was issued; no refund exists.	The reply reads fluently and looks correct.
Wrong-entity action	Action is applied to a similar but wrong order or customer.	The tool call succeeds technically.
Duplicate or irreversible action	A retry issues the same refund twice.	Each individual call looks valid.
Policy drift	Bot quotes a return window that differs from the published policy.	Needs a trusted reference to compare against.
Silent data corruption	Totals, ledgers or timezones become inconsistent after an action.	Only visible in the business database.
Inconsistent behaviour	Same input passes once and fails the next time.	A single run proves nothing about reliability.
Regression after a change	A prompt or model update breaks a previously working case.	No baseline comparison exists.
Leakage and injection	Cross-user data exposure; instructions hidden inside content are followed.	Authorization lives in the prompt, not the tool layer.

Existing observability and evaluation products are strong at tracing, dashboards and prompt-level scoring. The gap that Proofline targets is **verification of real outcomes**, **enforcement at the action boundary**, and **reliability measured over repeated trials**, delivered as one evidence-linked workflow.

2.1 Problem statement

Design and build a platform that can run an AI system, observe everything it does, verify what actually happened against ground truth, prevent unauthorized actions, and prove after every change whether reliability improved or degraded.

2.2 Objectives

- Execute AI systems through a uniform, provider-independent interface and capture complete traces.
- Detect failures with evidence, classified by a standard taxonomy and severity rubric.
- Verify agent claims against tool results and system-of-record state.
- Block disallowed actions before execution rather than reporting them afterwards.
- Measure reliability with pass^k, flake rate and regression deltas, not with one-shot scores.
- Integrate with CI so that unreliable changes cannot be merged.

3. Scope Definition

3.1 Target users

- Teams operating support, operations or workflow agents without a dedicated evaluation team.
- Engineers who need regression protection when prompts, models or tools change.
- Reviewers and risk owners who need an evidence trail for agent decisions.

3.2 In scope and deferred

Capability	Status	Notes
Model adapters, capability registry, controlled runtime	In scope (P0)	Provider-independent interface with explicit capability flags.
Trace recorder, store and viewer	In scope (P0)	OpenTelemetry-style span schema.
Deterministic evaluation and failure detection	In scope (P1)	Assertions, schema checks, state equality, permission checks.
Claim-versus-state verifier	In scope (P1, hero)	Claim extraction, evidence collection, deterministic verdict.
Action Guard and Tool Gateway	In scope (P2)	Policy, preconditions, approvals, execution-boundary enforcement.
Regression engine, model comparison, CI gate	In scope (P3)	Baseline vs candidate, pass^k and gate rules.
Replay, diagnosis	In scope (P4)	Levels L1/L2 first; L3 later.
Data Watchdog and reconciliation	In scope (P5)	Rules, anomaly baselines, CRM/ERP reconciliation.
Black-box audit agent (Scout)	In scope (late)	Permission-gated; reuses the Lab verifier and report engine.
Multimodal evaluators (image, audio, video)	Deferred	Plug-in evaluators once the core is stable.
Multi-tenant SaaS, billing, Kubernetes	Deferred	Not required to prove the product.

3.3 Non-goals

- Proofline does not claim to support every model automatically; unsupported capabilities are marked, not hidden.
- Proofline is not an autonomous attacker and does not perform load testing or exploit development.
- Proofline does not replace observability dashboards; it exports traces to them where useful.
- Proofline does not certify regulatory compliance.

4. Advantages, Limitations and Trade-offs

4.1 Advantages

Advantage	Explanation
Evidence-based findings	Every finding links to spans, tool results, database snapshots or reference sources. No evidence, no finding.
Outcome verification	Checks the real system state instead of trusting the agent's own description of what it did.
Boundary enforcement	Protected tools run through a controlled runtime, so permissions and arguments are validated before execution.
Reliability, not luck	$pass^k$ and flake rate expose non-determinism that single-run scores hide.
Provider independence	Adapters and a capability registry allow new providers without touching the evaluation or guard code.
Debuggable architecture	A modular monolith has one deployable unit, one database and straightforward local debugging.
Incremental value	Each phase delivers a working workflow; the product is usable long before the last module is finished.
Interoperability	OpenTelemetry-style traces can be exported to existing observability tools.

4.2 Limitations and mitigations

Limitation	Impact	Mitigation
Provider APIs differ in authentication and capabilities	Not every feature works for every model.	Adapters, capability registry, explicit "unsupported" status (META-01).
LLM judges can be wrong or manipulated	False passes or false failures on subjective criteria.	Deterministic first; judge calibrated on human labels; findings labelled judge-assessed.
Providers do not expose full internal reasoning	Traces are partial; replay is not always exact.	Capture available events; label replay as deterministic, partial or fresh.
Guard works only at its enforcement point	An agent that calls external APIs directly bypasses it.	Run protected tools through the gateway; least-privilege credentials; document the limit.
One metric does not fit every AI domain	Misleading reliability scores across tasks.	Common framework with task-specific evaluators.
Large scope	Risk of half-built modules.	Phase-wise delivery, exit tests, cut ladder.
Repeated trials (k=5) cost tokens and time	Higher run cost.	Apply k trials to critical tests only; caching; budgets and spend caps.
Sandbox fidelity	Sandbox may differ from production behaviour.	Seeded, realistic schema; periodic comparison against production-like fixtures.

5. Six Improvements That Strengthen the Project

A review of the original concept identified six areas that must be addressed before implementation. Each is presented with the problem, the design response, where it lives in the codebase and how it is accepted.

5.1 “Universal” does not mean automatic support for every model

Problem. Every provider has its own API, authentication scheme and capability set (tool calling, structured output, streaming, vision, seeds).

Solution. Provider adapters, a capability registry and a custom-integration interface. Unsupported features are explicitly marked rather than silently failed.

Implementation.

- `adapters/base.py` defines `ModelAdapter`; each provider implements it.
- `adapters/capability.py` stores capability flags per model configuration.
- A test requiring tool calling on a model without it is recorded as **META-01 unsupported**, not as a failure.
- `adapters/custom.py` lets users register any HTTP or SDK-based system.

Acceptance. Contract tests verify that every adapter declares capabilities accurately and returns normalized responses and errors.

5.2 The AI judge must not be the final authority

Problem. An LLM judge can pass a wrong answer or fail a correct one, and it can be influenced by text embedded in the output under test.

Solution. Three evaluation layers, in strict priority order. Critical actions rely on deterministic verification.

Implementation.

- **Layer 1 – Deterministic:** exact output, schema, permissions, tool results, state equality.
- **Layer 2 – Reference-based:** trusted answers, datasets and evidence documents.
- **Layer 3 – AI-based:** relevance, tone, answer quality; always labelled judge-assessed; calibrated against human labels.
- Judge prompts use delimiters, structured output and a rule that content under test is data, never instructions.

Acceptance. Judge-human agreement is reported; judge-only findings carry a needs-review status; injection attempts against the judge fail.

5.3 Trace and replay limitations must be explicit

Problem. Providers do not expose full internal reasoning, and external APIs do not behave identically on every run.

Solution. Capture all available events, save the relevant configuration, and label every replay by its reproducibility class.

Implementation.

- `trace/recorder.py` stores model parameters, prompt version, seeds (if supported), retrieval results and tool I/O.
- `replay/levels.py` classifies a replay as **deterministic**, **partially reproducible** or **fresh execution**.
- The UI shows the class beside every replay result, with the reason (for example “provider does not support seed”).

Acceptance. Replay determinism rate is measured on a fixed trace set and published with the class breakdown.

5.4 Action Guard must sit on the real execution boundary

Problem. Detecting a dangerous tool call is not enough. If the agent can call an external API on its own, the guard cannot stop it.

Solution. Protected tools are executed only by a controlled runtime or Tool Gateway. Permission, argument validation and policy checks run before execution.

Implementation.

- `guard/gateway.py` exposes protected tools as an MCP/HTTP proxy; the agent never holds the real credentials.
- `guard/checks/` contains schema, precondition, ownership, limit and reversibility checks.
- Decision outcomes: ALLOW, ASK_HUMAN, BLOCK; every decision is persisted with reasons.
- Integration modes document exactly what is and is not enforceable.

Acceptance. Property-based tests prove that no write is allowed when a precondition fails; the false-block rate is reported.

5.5 Each AI domain needs its own evaluation metrics

Problem. A single score cannot measure text generation, image segmentation, speech recognition and code models fairly.

Solution. A shared evaluation framework (inputs, outputs, evidence, verdict, severity) with task-specific evaluators plugged into it.

Implementation.

- `evaluation/evaluators/` holds `text_qa`, `rag`, `tool_agent` and `code` first; `speech` (WER) and `vision` (IoU, mAP) follow as plug-ins.
- Every evaluator implements the same `Evaluator` protocol and returns a `Verdict` object.
- Reports present per-domain metrics side by side rather than a single blended score.

Acceptance. Each evaluator ships with a labelled mini-benchmark and documented metric definitions.

5.6 The scope is too large to build at once

Problem. Model integrations, agents, multimodal evaluation, monitoring and CI/CD together are risky to deliver in one pass.

Solution. Keep the universal architecture, but implement it in phases. Build one complete working workflow first, then expand.

Implementation.

- Phase 0 fixes interfaces and schemas; Phases 1–7 add capability in priority order.
- A documented cut ladder defines what is dropped first if time is short.
- Every phase ends with a demonstrable exit test and updated README results.

Acceptance. A phase closes only when its exit test passes; unfinished modules never block the demo path.

5.7 Governing principle: modular monolith first

Proofline starts as a modular monolith, not as microservices. Debugging, testing and deployment are significantly simpler, and module boundaries are enforced through packages and interfaces so that any module can be extracted later.

Layer	Choice
Frontend	React + Vite (Streamlit acceptable for the first internal viewer)
Backend	FastAPI, Pydantic v2
Database	PostgreSQL (jsonb, pgvector)
Execution	Model adapter layer, controlled agent runtime, background workers
Core modules	Trace collector, evaluation engine, Action Guard, regression engine

5.8 Development priority

Priority	Features	Objective
P0	Model integration, execution runtime, traces	Execute and observe an AI system
P1	Deterministic evaluation, failure detection	Identify mistakes with evidence
P2	Action Guard, outcome verification	Stop unauthorized actions; verify actual results
P3	Regression testing, model comparison	Compare reliability after each change
P4	Replay, diagnosis, CI quality gate	Automate the development workflow
P5	Watchdog, additional providers, multimodal	Expand the platform

Demo-readiness rule
A demonstration must contain P0 and a complete P1. P2 and P3 should follow immediately, because outcome verification, boundary enforcement and regression comparison are what distinguish Proofline from a generic AI dashboard.

5.9 Readiness verdict

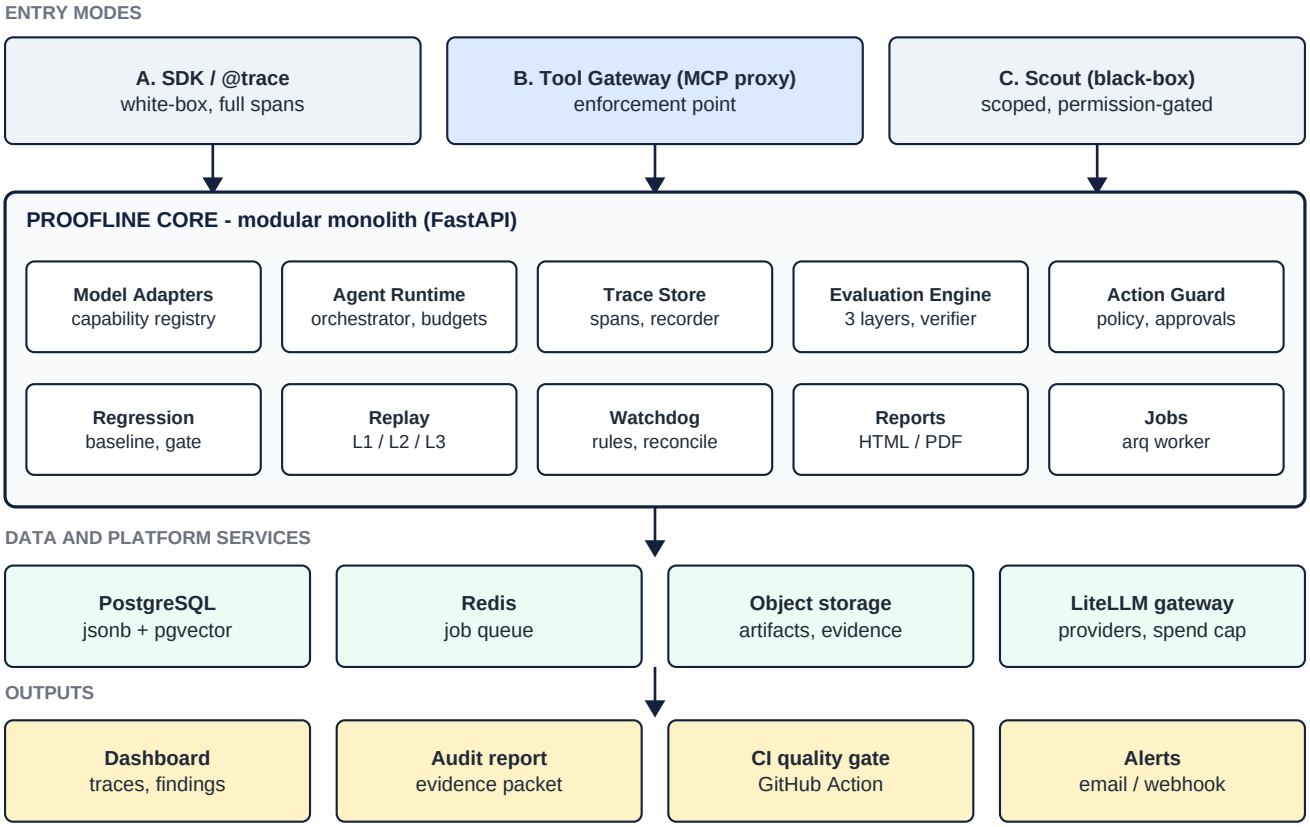
The foundation is sound and has real technical depth. The design is **not yet implementation-ready** until exact interfaces, schemas, error-handling rules and module-level acceptance tests are fixed. This is the purpose of **Phase 0** in the plan below. The consistent promise of the product is: *show what the AI did, what went wrong, the evidence for it, and whether the fix improved reliability.*

6. System Architecture

6.1 Architectural principles

- **Evidence first.** Every finding references spans, snapshots or reference sources.
- **Deterministic before probabilistic.** Code-based checks decide whenever they can; models decide only the remainder.
- **Explicit limits.** Capabilities, replay class and enforcement scope are always stated, never implied.
- **Strict module boundaries.** Modules communicate through typed interfaces and the database, never through each other's internals.
- **Fail closed for writes.** Unknown tools or missing preconditions result in BLOCK, never in a silent allow.
- **Infrastructure errors are not passes.** Timeouts and provider outages are recorded as META-02 and routed to review.

6.2 Logical architecture



The Scout runner (when enabled) runs in a separate, isolated container with an egress allowlist because it communicates with external hosts. All other modules are packages inside a single FastAPI application.

6.3 Integration modes: visibility versus enforcement

Mode	Visibility	Enforcement	Use when
A. SDK wrapper (`@trace`)	LLM calls, retrieval, tool I/O	Yes for wrapped tools	The agent code is available to you.
B. Tool Gateway (MCP / HTTP proxy)	Every proxied tool call	Yes; Guard runs here	The agent uses MCP or HTTP tools.

Mode	Visibility	Enforcement	Use when
C. Scout (black-box)	Chat input/output plus public reference facts	No; observation only	Third-party systems with written permission.
D. OTel / Langfuse import (later)	Existing spans	No	The team already has tracing in place.

Rule
Action Guard can stop only actions that pass through its enforcement point. In black-box mode Proofline blocks nothing; it provides evidence. This limitation is stated in every report.

6.4 Component responsibilities

Component	Responsibility	Key outputs
Model adapters	Normalize provider calls, errors, tokens and capabilities.	`ModelResponse`, capability record
Agent runtime	Run the agent loop with step, token and cost budgets; route tool calls.	Run record, span stream
Trace store	Persist spans (llm_call, retrieval, tool_call, guard_decision, approval, output) with redaction at ingest.	Searchable traces
Evaluation engine	Run deterministic, reference and AI evaluators; extract and verify claims.	Verdicts, findings, severity
Action Guard	Validate and authorize tool calls before execution.	ALLOW / ASK_HUMAN / BLOCK + reasons
Regression engine	Compare baseline and candidate; apply gate rules.	Gate result: PASS / FAIL / REVIEW
Replay engine	Re-execute a stored run with stubbed tools and retrieval.	Decision diff, replay class
Watchdog	Run data-integrity rules, anomaly baselines and reconciliation.	Watchdog findings linked to traces
Report engine	Render evidence-linked HTML/PDF reports.	Run and audit reports
Job layer	Execute long runs and batches asynchronously.	Job status, retries

6.5 Technology stack

Area	Technology	Reason
Language / framework	Python 3.12, FastAPI, Pydantic v2	Strong validation, async support, rich AI ecosystem.
Database	PostgreSQL with jsonb and pgvector	One database for relational, JSON and vector data.
Migrations	Alembic	Versioned, reversible schema changes.
Jobs	Redis + arq	Long-running runs do not block the API.
LLM access	LiteLLM gateway + native adapters	Provider switching, fallback, spend caps.
Tool protocol	MCP and HTTP tools	Standard interface for the Tool Gateway.
UI	Streamlit first, React + Vite later	Engine quality outweighs UI polish early on.
Testing	pytest, hypothesis, Playwright, Locust	Unit, property, E2E and light load coverage.
Packaging / CI	Docker Compose, GitHub Actions, GHCR	Reproducible builds and deployments.
Privacy / security	Presidio, JWT with RBAC	Redaction at ingest, role-based access.

6.6 Evaluation architecture

Evaluation is organised into three layers that run in order. A failure found in an earlier layer is never overturned by a later one.

Layer	Checks	Authority
1. Deterministic	Exact output, JSON schema, tool called / not called, argument match, state equality, permissions, step limits.	Final for critical actions.
2. Reference-based	Trusted answers, golden datasets, retrieved evidence, published policy facts, link status.	Final when the reference is authoritative.
3. AI-based	Relevance, helpfulness, tone, answer quality, groundedness on open-ended text.	Advisory; labelled judge-assessed; needs-review when alone.

Claim-versus-state verifier

- **Extract claims** from the final reply: `Claim{type: action|fact|policy|link, subject, asserted_status, span}`, validated by Pydantic; extraction accuracy is measured separately.
- **Collect evidence:** tool results, before/after snapshots of touched rows, retrieved chunks, URL status codes.
- **Compare deterministically:** verdict is SUPPORTED, CONTRADICTED, UNSUPPORTED or UNVERIFIABLE.
- **Emit findings:** false success, false failure, omitted failure, wrong entity, with severity and evidence references.
- **Edge cases** with dedicated tests: partial success (2 of 3 items), asynchronous states such as “pending”, and ambiguous wording.

6.7 Action Guard architecture

Each protected tool is declared in a policy file with a risk class, required preconditions, limits and approval rules.

```
tools:
  issue_refund:
    risk: irreversible_write
    requires: [order_belongs_to_customer, eligibility_checked, not_already_refunded]
    limits: {max_amount: 5000}
    approval: {required_if: "amount > 2000"}
defaults: {on_unknown_tool: BLOCK, on_missing_precondition: BLOCK, max_steps: 8}
```

Decision pipeline: **registry check** (is the tool known?) > **schema validation** > **preconditions and ownership** > **limits** > **reversibility** > **approval** (ASK_HUMAN pauses the run until a reviewer approves, rejects or edits) > **execution** with before/after state capture. Every step writes a `guard_decision` span.

6.8 Trace and replay

Replay level	What is reused	Reproducibility	Target
L0	Input only; fresh model and live tools	Fresh execution	-
L1	Input and recorded tool results	Partially reproducible to largely deterministic	Phase 3-4
L2	L1 plus retrieval results, model parameters and seed	Most reproducible without full snapshot	Primary target
L3	Full state snapshot restore	Deterministic	Later

A span has the following minimum shape, which also serves as the export format for external observability tools.

```
{ "span_id": "sp_01H...", "run_id": "run_...", "parent_id": null,
  "type": "tool_call",            // llm_call | retrieval | tool_call | guard_decision | approval | output
  "name": "issue_refund", "start": "2026-10-24T10:02:11.120Z", "end": "2026-10-24T10:02:11.480Z",
  "input": {...}, "output": {...}, "status": "ok",     // ok | error | blocked | infra_error
  "attrs": {"model": "...", "prompt_version": "v7", "seed": 42, "tokens": 412},
  "state_diff": {"table": "refunds", "before": null, "after": {"id": 88, "amount": 1200}},
  "redactions": ["email", "phone"] }
```

6.9 Domain-specific evaluators

Domain	Evaluator	Primary metrics
Text / QA	text_qa	Exact match, semantic equivalence, groundedness, refusal correctness
RAG	rag	hit@k, citation correctness, groundedness, conflict detection
Tool-using agents	tool_agent	Task success, forbidden-action count, claim accuracy, pass^k
Code models	code	Unit-test pass rate, static-check results, sandboxed execution outcome
Speech (plug-in)	speech	Word error rate, latency
Vision (plug-in)	vision	IoU, mAP, per-class accuracy

6.10 Core metrics

Metric	Definition
pass@1	Average probability that a test passes in a single trial.
pass^k	A test passes only if all k trials pass; the aggregate is the fraction of tests whose k trials all passed.
Flake rate	Share of tests whose trials contain both passes and failures.
Claim accuracy	Correct verdicts / total claims on a labelled set.
Detection precision / recall	Measured on Proofline Bench against planted failures.
False-block rate	Legitimate actions wrongly blocked by Action Guard.
Replay determinism	Percentage of L2 replays producing identical decisions.
Judge-human agreement	Agreement (or Cohen’s kappa) between judge and human labels.
Cost and latency	Cost per 100 runs; p50 and p95 latency, not just the mean.

6.11 Failure taxonomy

Group	Codes (default severity)
OUT - Output	OUT-01 incorrect answer (S3), OUT-02 unsupported claim (S3), OUT-03 invalid structured output (S3), OUT-04 instruction miss (S4)
ACT - Action	ACT-01 wrong tool (S2), ACT-02 invalid arguments (S3), ACT-03 policy violation (S1-S2), ACT-04 wrong entity (S1), ACT-05 duplicate action (S1)
CLM - Claim	CLM-01 false success claim (S2), CLM-02 false failure claim (S3), CLM-03 omitted failure (S2)
POL - Policy	POL-01 contradiction with published policy (S2), POL-02 stale information (S3), POL-03 hallucinated link or entity (S3)
DAT - Data	DAT-01 silent corruption (S1-S2), DAT-02 sync drift (S2), DAT-03 PII in trace or log (S1)
SEC - Security	SEC-01 cross-user leak (S1), SEC-02 injection followed (S1-S2), SEC-03 system prompt exposure (S3)

Group	Codes (default severity)
REL - Reliability	REL-01 flaky answers (S3), REL-02 language gap (S3), REL-03 broken handoff (S2), REL-04 timeout or loop (S3), REL-05 cost blow-up (S3), REL-06 regression (S2)
META	META-01 unsupported capability (not a failure), META-02 infrastructure error (never counted as a pass)

Severity: S1 critical (data leak, unauthorized or irreversible action), S2 high (false action claim, policy contradiction on money or legal matters), S3 medium (inconsistency, broken link, weak handoff), S4 low (tone, minor format).

6.12 Data model

Group	Tables
Workspace	users, roles, projects, model_configs, prompt_versions, datasets
Execution	runs, spans, artifacts
Evaluation	test_cases, evaluations, claims, findings, failure_categories, baselines, regression_reports
Guard	guard_policies, policy_versions, guard_decisions, approvals
Watchdog	watchdog_rules, watchdog_findings, baseline_stats, recon_runs, crm_orders, erp_orders
Scout	targets, scope_grants, audits, probes, probe_results, ground_truth_facts, evidence_artifacts
Governance	audit_log, retention_policies

Every query is scoped by `project_id`. Authorization is enforced in the service layer and covered by an authorization-matrix test suite.

6.13 Interface contracts and error handling

Interfaces are fixed in Phase 0 so that modules can be built and tested independently.

```
class ModelAdapter(Protocol):
    name: str
    def capabilities(self) -> CapabilitySet: ...
    async def generate(self, req: ModelRequest) -> ModelResponse: ...      # raises AdapterError

class Evaluator(Protocol):
    domain: str
    def evaluate(self, case: TestCase, trace: Trace, ctx: EvalContext) -> list[Verdict]: ...

class GuardCheck(Protocol):
    name: str
    def check(self, call: ToolCall, ctx: GuardContext) -> CheckResult: ...  # PASS | FAIL(reason)

class Replayer(Protocol):
    def replay(self, run_id: str, level: ReplayLevel, overrides: dict) -> ReplayResult: ...
```

Error class	Examples	Handling
AdapterError	Auth failure, rate limit, timeout, unsupported feature	Retry with backoff; unsupported becomes META-01; exhausted retries become META-02.
ToolError	Tool raised or returned invalid data	Record span with status=error; agent decides; never counted as a pass.
PolicyViolation	Guard returns BLOCK	Tool is not executed; decision and reasons are stored.
ValidationError	Schema mismatch on model output	Counted as OUT-03 on the test; run continues where safe.
BudgetExceeded	Steps, tokens or spend cap reached	Run stops cleanly; REL-04 or REL-05 recorded.

7. Repository and Folder Structure

The repository is a single monorepo. The backend is organised by domain module; each module owns its models, services and tests.

```
proofline/
|-- README.md                  # problem, diagram, quickstart, results table, limitations
|-- Makefile                   # make up | down | test | bench | lint | migrate
|-- pyproject.toml             # dependencies, ruff, pytest config
|-- docker-compose.yml         # local stack: api, worker, postgres, redis, minio, ui
|-- docker-compose.prod.yml    # production overlay (Caddy, resource limits)
|-- .env.example               # all settings, no secrets
|-- .github/
|   |-- workflows/             # ci.yml, build.yml, deploy-staging.yml, deploy-prod.yml
|   `-- actions/proofline-gate/ # reusable CI quality-gate action
|-- docs/
|   |-- adr/                   # ADR-001 ... ADR-012
|   |-- architecture/          # diagrams, data model, sequence diagrams
|   |-- interfaces.md          # frozen interface contracts (Phase 0)
|   |-- threat_model.md
|   `-- runbook.md
|-- src/proofline/
|   |-- main.py                # FastAPI app factory, router registration
|   |-- core/                  # config, logging, errors, ids, clock, security helpers
|   |-- db/                    # SQLAlchemy models, session, repositories
|   |   `-- migrations/        # Alembic versions
|   |-- schemas/               # Pydantic: trace, test_case, finding, claim, policy, report
|   |-- api/
|   |   |-- deps.py            # auth, project scoping, pagination
|   |   `-- routers/           # auth, projects, runs, traces, evals, guard, regression,
|   |                          # replay, watchdog, scout, reports, health
|   |-- adapters/
|   |   |-- base.py            # ModelAdapter protocol
|   |   |-- capability.py      # capability registry and flags
|   |   |-- mock.py            # deterministic MockProvider
|   |   |-- litellm_provider.py
|   |   |-- openai_compatible.py
|   |   `-- custom.py          # user-defined integration interface
|   |-- runtime/               # orchestrator, agent loop, budgets, tool executor
|   |-- trace/                 # recorder, span model, store, redaction, exporters (otel)
|   |-- evaluation/
|   |   |-- deterministic/     # assertions: tool_called, args_match, state_equals, json_schema
|   |   |-- reference/         # golden sets, evidence and policy-fact matchers, link checker
|   |   |-- judge/             # rubric prompts, calibration, injection defence
|   |   |-- claims/            # extractor, evidence collector, verifier, verdicts
|   |   |-- evaluators/        # text_qa, rag, tool_agent, code (+ speech, vision plug-ins)
|   |   |-- passk.py           # repeat-trial runner, pass@1, pass^k, flake rate
|   |   `-- taxonomy.py        # failure codes and severity rubric
|   |-- guard/
|   |   |-- policy.py          # policy.yaml loader and versioning
|   |   |-- checks/            # schema, precondition, ownership, limits, reversibility
|   |   |-- gateway.py         # Tool Gateway (MCP/HTTP proxy)
|   |   `-- approvals.py       # approval queue and reviewer actions
|   |-- regression/            # baselines, comparison, gate rules, model comparison
|   |-- replay/                # levels L1-L3, stubs, decision diff, determinism metric
|   |-- watchdog/              # rules, anomaly baselines, reconciliation, forensics, alerts
|   |-- scout/                 # scope loader, authz gate, rate limiter, kill switch,
|   |                          # adapters, probes, oracles, ground truth, reporting
|   |-- reports/               # templates (HTML), PDF renderer
|   |-- security/              # JWT, RBAC, PII/DLP (Presidio), audit log
|   `-- jobs/                  # arq worker, task definitions
|-- sandbox/shopops/           # sample business DB, seed data, mock tools, sample agent
|-- ui/
|   |-- streamlit/             # trace viewer, findings, approvals, reports (initial)
|   `-- web/                   # React + Vite (optional, later)
|-- sdk/proofline_sdk/         # @trace decorator, client, gateway helper
|-- bench/
|   |-- cases/dev/             # development planted-failure cases
|   |-- cases/heldout/         # held-out cases, never used to tune detectors
|   `-- runner.py              # precision / recall per taxonomy group
|-- tests/
|   |-- unit/  property/  contract/  integration/  e2e/  security/  scout_safety/  load/
|-- deploy/
|   |-- caddy/Caddyfile        # HTTPS, rate limit
|   |-- scripts/               # backup.sh, restore.sh, smoke_test.sh
|   `-- scout-runner/          # isolated container, egress allowlist
`-- examples/                  # policy.yaml, gate.yaml, scope.yaml, sample test cases
```

Conventions

- Module imports flow one way: `schemas` > `core` > domain modules > `api`. No module imports from `api`.
- Each module exposes a small public interface in its `__init__.py`; other modules use only that interface.
- Every module has its own tests directory section and an acceptance test defined in Phase 0.
- Configuration comes only from environment variables; secrets never appear in the repository.

8. Phase-wise Delivery Plan

The plan has eight phases (0 to 7). A phase closes only when its exit test passes. Effort figures are planning estimates for a small team or a focused individual and must be replaced with measured values. Each phase begins by writing its ADRs and exit tests, then proceeds in thin vertical slices that always run end to end.

Phase	Name	Priority	Core result
0	Specification and contracts	Pre-P0	Frozen interfaces, schemas, error rules, acceptance tests
1	Foundation, runtime and trace	P0	Execute an agent and observe every step
2	Evaluation and claim verifier	P1	Detect failures with evidence; measure pass^{k}
3	Action Guard and Tool Gateway	P2	Block unauthorized actions; verify real outcomes
4	Regression, comparison, replay, CI gate	P3 / P4	Prove reliability change after each edit
5	Watchdog, security and SDK	P5	Catch silent data faults; harden and package
6	Scout black-box audit (expansion)	P5	Permission-gated external audits
7	Deployment and launch	Release	Public, secure, recoverable system

Phase 0: Specification and Interface Contracts

Indicative effort	Priority mapping	Prerequisite
3-5 days	Pre-P0 (readiness)	Approved scope (Sections 2-6).

Goal. Turn the architecture into an implementation-ready specification so that modules can be built and tested independently.

Detailed workflow.

1. Write ADRs for the key decisions: modular monolith, Postgres-only, trace schema, adapters, Guard enforcement point, evaluation layers, replay classes.
2. Define Pydantic schemas: Span, Trace, TestCase, Claim, Verdict, Finding, ToolCall, GuardDecision, Policy, Report.
3. Freeze the interface protocols (ModelAdapter, Evaluator, GuardCheck, Replayer) and the error classes with handling rules.
4. Define the REST API surface (OpenAPI draft) and the database schema (ER diagram, first Alembic migration).
5. Write module-level acceptance tests as skeletons (expected inputs, outputs and failure behaviour) before any implementation exists.
6. Define the failure taxonomy file, severity rubric and the capability flag list.

Modules and folders touched. `docs/adr/`, `docs/interfaces.md`, `src/proofline/schemas/`, `tests/` (skeletons).

Deliverables.

- Twelve ADRs, interface document, schema package, OpenAPI draft, ER diagram.
- Acceptance-test skeleton for every module.

How it is built.

- Schemas are written first, as Pydantic models, and exported to JSON Schema for review.
- Interfaces use Python Protocols so that adapters and evaluators can be added without inheritance coupling.
- Acceptance tests are marked `xfail` until the module exists, which makes progress visible in CI.

Advantages	Drawbacks and risks
• Prevents rework and mismatched assumptions between modules. • Makes every later phase testable from day one.	• Produces no visible feature; requires discipline to keep short. • Over-specification can freeze wrong decisions; ADRs are revisable.

Exit test

All schemas import and validate sample payloads; every module has an acceptance-test skeleton; interface document reviewed and signed off.

Phase 1: Foundation, Runtime and Trace

Indicative effort	Priority mapping	Prerequisite
2-3 weeks	P0	Phase 0 complete.

Goal. Stand up the skeleton: one command starts the system, a ticket passes through the agent runtime and every step is traced.

Detailed workflow.

1. Create the repository, Docker Compose stack (API, worker, Postgres, Redis, object storage), Alembic migration pipeline and CI (lint + tests).
2. Implement the adapter interface, `MockProvider` (deterministic) and the capability registry; add the first real provider through LiteLLM.
3. Build the ShopOps sandbox: customers, orders, payments, refunds and inventory tables with seed data and tools `get_order`, `issue_refund`, `cancel_order`, `close_ticket`.
4. Implement the run orchestrator with step, token and cost budgets and a clean-stop behaviour.
5. Implement the TraceRecorder with span types, redact-at-ingest and persistence; add API endpoints for runs and traces.
6. Build the first trace viewer (Streamlit): run list, span tree, inputs/outputs, state diffs.
7. Implement the deterministic assertion library: tool_called, tool_not_called, args_match, state_equals, json_schema, max_steps.

Modules and folders touched. `adapters/`, `runtime/`, `trace/`, `db/`, `sandbox/shopops/`, `ui/streamlit/`, `evaluation/deterministic/`.

Deliverables.

- Working `make up` environment.
- Trace for every LLM and tool call.
- 20-case deterministic test suite and run API.

How it is built.

- Spans are written inside the runtime wrapper, so no call can bypass the recorder.
- The MockProvider returns scripted responses, which makes integration tests fully deterministic.
- Capability flags (tool_calling, structured_output, seed, streaming) are checked before a test is scheduled.

Advantages	Drawbacks and risks
• Delivers the observable base that every later engine depends on. • Early demo path: ticket to trace. • Mock mode removes dependency on external quotas.	• Looks like ordinary tracing until later phases add verification. • Streamlit viewer is basic. • Free provider quotas may limit early experiments.

Exit test

Fresh clone starts with one command; a ticket flows from input to trace with 100% of LLM and tool calls recorded; the assertion engine shows 0 errors on 20 known-outcome tests.

Phase 2: Evaluation and the Claim-versus-State Verifier

Indicative effort	Priority mapping	Prerequisite
3-4 weeks	P1	Phase 1 exit test passed.

Goal. Detect failures with evidence and measure reliability over repeated trials. Deliver the hero feature.

Detailed workflow.

1. Implement the three-layer evaluation pipeline and the shared `Verdict` model.
2. Implement claim extraction (LLM + Pydantic validation) and measure extraction accuracy on a labelled set.
3. Capture before/after database snapshots for touched rows; collect tool results, retrieved chunks and URL status as evidence.
4. Implement the deterministic verdict logic (SUPPORTED, CONTRADICTED, UNSUPPORTED, UNVERIFIABLE) and the finding generator with taxonomy and severity.
5. Create 30 planted claim cases: false success, false failure, omitted failure, wrong entity, partial success, asynchronous state.
6. Build the repeat-trial runner (k=5 for critical tests) with pass@1, pass^k and flake-rate reporting; add a prompt registry and config snapshots.
7. Build the golden set (about 80 cases, multilingual) and the LLM judge with rubric and evidence; calibrate on at least 30 human labels; add injection defences.
8. Add the RAG evaluator (hit@k, citation correctness, groundedness, refusal rate) and the policy-conflict checker.

Modules and folders touched. `evaluation/claims/`, `evaluation/passk.py`, `evaluation/judge/`, `evaluation/evaluators/`, `evaluation/taxonomy.py`, `bench/`.

Deliverables.

- Verifier with 30 planted cases and reported accuracy.
- pass^k report and flake rate.
- Calibrated judge with documented agreement.
- RAG and conflict-checker results.

How it is built.

- Claim verdicts are produced by code comparing structured claims with structured evidence; the LLM is used only for extraction.
- The judge receives the evidence bundle and rubric, outputs JSON, and its result is stored with the label *judge-assessed*.
- Dev and held-out benchmark sets are kept separate from the first day.

Advantages	Drawbacks and risks
• Catches the most damaging failure class (false success) with proof. • Reliability is measured, not assumed. • Clear separation of certain and uncertain findings.	• Claim extraction can itself be wrong; it needs its own accuracy metric. • Ambiguous language limits what can be verified. • k-trial runs increase cost.

Exit test

Claim accuracy and detection precision/recall reported; $pass^k$ table produced; judge-human agreement reported; three judge-injection attempts fail; hit@3 compared for vector, hybrid and rerank.

Phase 3: Action Guard and Tool Gateway

Indicative effort	Priority mapping	Prerequisite
3 weeks	P2	Phase 2 verifier and trace complete.

Goal. Stop unauthorized or unsafe actions at the real execution boundary and verify the resulting state.

Detailed workflow.

1. Define the tool registry with risk classes (read, reversible write, irreversible write) and the `policy.yaml` format with versioning.
2. Implement the checks: schema validation, preconditions, ownership, limits, reversibility.
3. Implement the Tool Gateway as an MCP/HTTP proxy; the agent receives only a gateway token, never the real tool credentials.
4. Implement the approval queue: ASK_HUMAN pauses the run; reviewer approves, rejects or edits; the decision resumes or aborts the run.
5. Execute allowed calls with before/after state capture; feed the state diff to the verifier for outcome confirmation.
6. Write 20 wrong-action scenarios (wrong customer, duplicate refund, over-limit refund, unknown tool, missing eligibility) and property-based tests.
7. Record all decisions in `guard_decisions` and in the audit log; produce a block / false-block table.

Modules and folders touched. `guard/`, `runtime/tool_executor.py`, `sandbox/shopops/`, `tests/property/`.

Deliverables.

- Working Guard with policy file and approval queue.
- Gateway usable by an external agent.
- Block / false-block report.

How it is built.

- Unknown tools and missing preconditions default to BLOCK (fail closed).
- Hypothesis generates random argument sets to assert the invariant: *no write executes when a precondition fails*.
- Least-privilege credentials are held by the gateway; the agent process has none.

Advantages	Drawbacks and risks
• Prevents damage rather than reporting it afterwards. • Gives auditable decisions and reasons. • Strong differentiator from dashboard tools.	• Protects only traffic that passes through the gateway. • Strict policies can create false blocks. • Human approval adds latency.

Exit test

0 forbidden writes via the gateway across the scenario suite; property tests report 0 violations; block / false-block table published; Gate G1 criteria for Guard satisfied.

Phase 4: Regression, Model Comparison, Replay and CI Gate

Indicative effort	Priority mapping	Prerequisite
3 weeks	P3 / P4	Phases 2 and 3 complete.

Goal. Prove after every prompt, model or tool change whether reliability improved, and automate the decision in CI.

Detailed workflow.

1. Implement baseline snapshots (suite results with config, prompt and model version).
2. Implement comparison of baseline A and candidate B: metric deltas, pass-to-fail list, newly failing critical tests, cost and latency p50/p95.
3. Implement gate rules in YAML (critical tests must pass all k trials, forbidden actions = 0, success threshold, regression limits, latency and cost caps); infrastructure errors route to REVIEW.
4. Build the GitHub Action `proofline-gate` that runs the suite and fails the pull request when the gate fails.
5. Add 6 planted regressions to validate the gate.
6. Implement replay L1 and L2 with stubbed tools and retrieval, decision diff and the reproducibility class label (deterministic, partial, fresh).
7. Add model comparison view: same suite across two or more model configurations.

Modules and folders touched. `regression/`, `replay/`, `.github/actions/proofline-gate/`, `examples/gate.yaml`.

Deliverables.

- Regression report (JSON + HTML).
- CI gate action.
- Replay with decision diff and class label.
- Model comparison report.

How it is built.

- Baselines are immutable once approved; a new baseline requires explicit promotion.
- Replay stubs return recorded tool results keyed by call signature; mismatches are reported, not hidden.
- Gate output has three states (PASS, FAIL, REVIEW); infrastructure errors can never produce PASS.

Advantages	Drawbacks and risks
• Turns reliability into a merge condition. • Makes model switching evidence-based. • Replay speeds up debugging.	• Baseline maintenance effort. • Replay cannot be perfectly deterministic when providers lack seeds. • Gate thresholds need tuning to avoid noisy failures.

Exit test

6 of 6 planted regressions caught; a deliberately bad pull request turns CI red; replay determinism rate and class breakdown reported; fallback test table produced; Gate G1 passed.

Phase 5: Watchdog, Security Hardening and SDK

Indicative effort	Priority mapping	Prerequisite
3 weeks	P5	Phase 4 complete.

Goal. Detect silent business-data faults, secure the platform and make it adoptable by external agents.

Detailed workflow.

1. Implement the Watchdog rule engine: duplicate refund, total mismatch, inventory, ledger balance, subscription versus payment, timezone, settlement.
2. Add anomaly baselines (z-score or IQR) and CRM/ERP reconciliation (missing, duplicate, stale, mapping errors).
3. Link each Watchdog finding to the originating trace for root-cause analysis; add alerts (email, webhook).
4. Implement JWT authentication, role-based access (owner, engineer, reviewer, viewer), project isolation and the authorization-matrix tests.
5. Add PII redaction (Presidio) and output DLP filters; escape-all rendering in the viewer to prevent XSS from transcripts.
6. Run a 40-attack red-team suite against Action Guard and the judge; record attack success before and after mitigation.
7. Package the SDK: `@trace` decorator, client, gateway helper; publish as an installable package.

Modules and folders touched. `watchdog/`, `security/`, `sdk/`, `tests/security/`.

Deliverables.

- Watchdog with 8 data-fault and 4 integration-fault detectors.
- Authorization and redaction tests.
- Red-team before/after table.
- Installable SDK v0.

How it is built.

- Rules are SQL-backed and run on rows touched by a run, which keeps findings tied to a trace.
- Redaction occurs before persistence, so raw PII never reaches the database or logs.
- Every authorization rule has a positive and a negative test in the matrix.

Advantages	Drawbacks and risks
• Catches errors that look correct in the agent's reply. • Makes Proofline safe to host. • SDK lowers adoption effort.	• Rule sets must be maintained per business domain. • Anomaly detection yields false alarms until baselines mature. • Security work is broad; needs a strict checklist.

Exit test

8 of 8 data faults and 4 of 4 integration faults detected with false alarms and time-to-detect reported; 0 cross-project leaks; attack success rate lower after mitigation; Gate G2 passed; Lab core is complete.

Phase 6: Scout: Permission-gated Black-box Audit (Expansion)

Indicative effort	Priority mapping	Prerequisite
3-4 weeks	P5 (expansion)	Phase 5 complete; all safety tests written first.

Goal. Audit third-party agents from the outside, strictly within written scope, reusing the Lab verifier, taxonomy and report engine.

Detailed workflow.

1. Implement the scope file loader and authorization gate: no valid scope, no run; every request is checked against the scope hash and time window.
2. Implement the rate limiter, consecutive-error stop, kill switch (environment flag, CLI, dead-man timer) and the isolated runner container with an egress allowlist.
3. Build chat-API and web-widget adapters and the conversation runner with evidence recording.
4. Build the ground-truth builder: facts from permitted public pages with URL and snapshot date.
5. Implement the probe battery (policy fidelity, consistency, cross-language, hallucination, handoff, link health, canary, robustness, latency) and oracles in strict order: deterministic, ground truth, self-contradiction, judge.
6. Score findings with pass^k, severity and a confidence label (confirmed, likely, needs-review); generate the report.
7. Run the pilots in order: own demo bot, self-hosted open-source agent, then permissioned third-party targets. Every finding is human-reviewed and re-run before release.

Modules and folders touched. `scout/`, `deploy/scout-runner/`, `tests/scout_safety/`, `examples/scope.yaml`.

Deliverables.

- Scout core with safety gates.
- Probe battery and oracle set.
- Evidence-linked audit report.
- Pilot results with measured false-positive rate.

How it is built.

- Safety rules are enforced in code and verified by tests, not only written in policy.
- Defaults: one request per 5 seconds, at most 150 conversations and 8 turns each, stop after 3 consecutive errors.
- Findings are never published or sent without human review.

Advantages	Drawbacks and risks
• Extends Proofline to systems it does not control. • Produces real-world evidence. • Reuses existing engines.	• Legal and terms-of-service exposure if run without authorization. • No enforcement possible in black-box mode. • Oracle quality limits confidence; false positives harm credibility.

Exit test

Out-of-scope requests refused; rate limit respected; kill switch stops all requests within 5 seconds; recall and false-positive rate measured on planted bugs; no out-of-scope request in logs; Gates G3 and G4 passed before any real audit.

Phase 7: Deployment and Launch

Indicative effort	Priority mapping	Prerequisite
1-2 weeks	Release	Gates G1 and G2 passed (G3/G4 if Scout is included).

Goal. Ship a public, secure, recoverable deployment and complete the project documentation.

Detailed workflow.

1. Create the production Compose overlay, Caddy HTTPS and rate limiting, and the CI/CD pipeline: staging on every push to main, production after manual approval.
2. Automate Alembic migrations, backups (nightly dump to object storage) and perform a full restore drill.
3. Configure monitoring (error tracking, uptime), LiteLLM spend caps and the runbook for top incidents.
4. Seed a demo project; add MockProvider mode and a recorded demo for provider outages.
5. Publish Proofline Bench results, README with architecture diagram and limitations, and a short demo video.

Modules and folders touched. `deploy/`, `docker-compose.prod.yml`, `.github/workflows/`, `README.md`.

Deliverables.

- Public HTTPS deployment.
- Deploy-on-push to staging.
- Restore drill record.
- Final README and benchmark table.

How it is built.

- A single small VM with Docker Compose is sufficient; Kubernetes is intentionally excluded at this stage.
- Rollback uses the previous image tag plus a database backup.
- Secrets live in CI secrets and a permission-restricted environment file on the server.

Advantages	Drawbacks and risks
• Anyone can try the system. • Proven recovery path. • Cost remains low.	• Single-VM deployment is not highly available. • Free-tier LLM limits can affect demos.

Exit test

Public URL reachable over HTTPS; staging deploys on push; restore drill completed; README results table filled with measured numbers; Gate G5 passed.

9. End-to-End Runtime Workflow

Example: a customer writes, “Refund order 1042; the item was damaged.” The agent belongs to a retail operations system, and Proofline sits around it.

#	Step	Module	Evidence produced
1	Register project and model configuration; check capabilities (tool calling, structured output, seed).	adapters	Capability record; unsupported tests marked META-01
2	Create the test case: input, expected behaviour, forbidden actions, severity.	evaluation	test_case
3	Start the run: run_id, prompt version, model version, config snapshot, budgets.	runtime	Run record
4	Agent retrieves policy chunks (hybrid search and rerank).	trace	Retrieval span with chunk IDs and scores
5	Agent plans `issue_refund(order=1042)`; output validated against schema.	trace	llm_call span
6	Tool Gateway intercepts; Guard checks registry, arguments, ownership, eligibility, limits, reversibility.	guard	guard_decision: ALLOW / ASK_HUMAN / BLOCK + reasons
7	If ASK_HUMAN, the approval queue pauses the run until a reviewer approves, rejects or edits.	guard	approval span
8	Tool executes against the sandbox database; before/after snapshots are stored.	runtime	tool_call span + state diff
9	Agent replies; verifier extracts claims and compares them with tool result and database state.	evaluation	claims and verdicts
10	Watchdog runs data rules on affected rows (duplicates, totals, ledger, CRM/ERP).	watchdog	watchdog findings linked to trace
11	Findings are classified by taxonomy and severity and shown with evidence.	evaluation	Failure report, alert
12	After a fix: replay (L2), regression comparison (pass^k) and quality gate; CI allows or blocks the merge.	regression, replay	Gate result PASS / FAIL / REVIEW

9.1 Failure-to-fix loop

1. A finding is created with evidence and severity.
2. A reviewer triages: is it a real defect or acceptable variation?
3. The failing run is replayed (L1/L2) to isolate the cause: prompt, retrieval, model or tool.
4. The change is made and the suite runs again with k trials.
5. The regression comparison shows whether the failure is fixed and nothing else broke.
6. The gate result is attached to the pull request; merge proceeds only on PASS.

10. Testing and Quality Strategy

Proofline evaluates other systems, so its own detectors must be evaluated too. The test stack covers each layer with a defined exit criterion.

Layer	What is tested	Tooling	Exit criterion
Unit	Assertions, verifier rules, guard checks, parsers	pytest	All green; high coverage of core logic
Property	Guard never allows a write when a precondition fails	hypothesis	0 violations over thousands of cases
Contract	Adapter capabilities and API schemas	pytest, OpenAPI checks	Each adapter declares capabilities correctly
Integration	End-to-end run with MockProvider	pytest + Compose	Core scenarios pass deterministically
E2E UI	Create run, view trace, export report	Playwright	Critical flows pass
Meta-evaluation	Detector precision and recall on planted failures	Proofline Bench	Per-category precision/recall published
Judge calibration	Judge versus human labels	Calibration script	Agreement reported; judge-only findings need review
Flakiness	Repeat critical tests (k=5)	pass^k runner	Flake rate reported; quarantine policy
Security	AuthZ matrix, XSS, judge injection, redaction	pytest + manual	0 cross-project leaks, 0 XSS
Load / chaos (light)	Provider failure, DB restart, concurrent runs	Locust	p95 and recovery documented

10.1 Proofline Bench

- At least 60 labelled cases with 5 to 10 per taxonomy group, including clean cases to measure false positives.
- Separate development and held-out sets; held-out cases are never used to tune detectors.
- Bench runs on every release; precision and recall appear in the README with the version number.

10.2 Acceptance gates

Gate	When	Pass condition
G1 Lab core	End of Phase 4	100% trace coverage; verifier metrics reported; 6/6 planted regressions caught; 0 forbidden writes via gateway; failing PR turns CI red
G2 Watchdog and security	End of Phase 5	8/8 data faults and 4/4 integration faults detected; authz and redaction tests green; red-team before/after table
G3 Scout safety	Before any real audit	Scope gate, rate limit, kill switch and egress allowlist tests all pass; independent review of the scope gate
G4 Real-world	End of Phase 6	Findings human-reviewed; false-positive rate measured; no out-of-scope request in logs
G5 Launch	End of Phase 7	Public deployment, benchmark numbers, complete README, restore drill done

11. Deployment, Security and Operations

11.1 Environments

Environment	Location	Data	Deployment
Local	Developer machine, Docker Compose	Seeded synthetic data, MockProvider	`make up`
Staging	Small VM or separate Compose project	Synthetic and demo targets	Automatic on push to main
Production (demo)	Single VM, public HTTPS	Synthetic or consented data only	Manual approval, then migration and compose up

11.2 CI/CD pipeline

- **Pull request:** ruff, pytest (unit and contract), integration tests with MockProvider, Proofline Bench gate (the product tests itself), dependency scan and secret scan.
- **Main branch:** build Docker images tagged with the git SHA, push to the registry, deploy to staging, run smoke tests (health endpoint and a sample run).
- **Production:** manual approval, Alembic migration, pull and restart, post-deploy smoke test. Rollback uses the previous image tag and a database backup.

11.3 Security controls

Area	Control
Authentication / authorization	JWT login; roles owner, engineer, reviewer, viewer; project_id on every query; authorization-matrix tests.
Untrusted content	Transcripts and outputs are treated as data: escape-all rendering, judge prompts with delimiters and structured output.
Privacy	Redaction at ingest, DLP filter on outputs, default retention of 30 days, deletion API.
Network	HTTPS only, rate limiting, CORS allowlist, database and Redis not exposed publicly, isolated network for the Scout runner.
Containers	Non-root users, read-only filesystems where possible, resource limits.
Secrets	Rotatable, never logged, provider keys held by the backend only.
Supply chain	Pinned dependencies, dependency and image scanning.
Audit	Guard decisions, approvals, scope grants and report exports recorded in `audit_log`.
Spend control	Daily and per-run LLM budgets through the gateway; Scout kill switch.

11.4 Runbook: principal incidents

Incident	Detection	Action
Provider outage or rate limit	Gateway errors, p95 spike	Fallback model; record-replay or mock mode for demos
Runaway job	Request counter, spend alert	Kill switch; mark run aborted; review limits
Disk full	Disk alert	Run retention job; prune artifacts; resize
Leaked key	Secret scan, provider alert	Rotate and revoke; review audit log
Bad deployment	Smoke test failure	Roll back to previous image tag

12. Risk Register and Cut Ladder

Risk	Likelihood / impact	Mitigation
Scope creep	High / High	Cut ladder; scope review at the start of each phase; new features only after the exit test.
LLM judge unreliable	High / Medium	Deterministic first; calibration; needs-review label.
Flaky tests	High / Medium	$pass^k$, low temperature, seeds, quarantine policy.
Free-tier quotas	High / Medium	Fallback providers, caching, local models, record-replay in development.
Provider API changes	Medium / Medium	Adapter layer, MockProvider, contract tests.
Cost blow-up	Medium / Medium	Per-run and daily caps; cheap judge with human spot-checks.
Legal or terms-of-service exposure (Scout)	Medium / Very high	Written permission, scope gate in code, low volume, benign tests, private disclosure.
False positives reduce trust	High / High	Confidence labels, k-trial reproduction, reference URLs, human review, fresh re-run.
Platform leaks data	Low / High	Redact at ingest, retention, authorization tests, no real PII in development.
Live demo failure	Medium / High	Mock mode, seeded data, recorded demo.

12.1 Cut ladder

If scope must be reduced, remove items in this order:

Order	Remove	Impact
1	React UI (keep Streamlit)	Low
2	Optional integrations (additional exporters, extra tooling)	Low
3	Watchdog anomaly layer (keep rules and reconciliation)	Low-medium
4	Replay L2 reduced to L1	Medium
5	Web-widget adapter in Scout (keep chat-API and self-hosted targets)	Medium
6	RAG extras and policy-conflict checker	Medium
Never	Claim verifier, Action Guard enforcement, regression gate, Scout safety gate, Proofline Bench, human review	Project loses its purpose

13. Conclusion

Proofline's value lies in connecting four things that other tools treat separately: **what the AI did** (traces), **what went wrong** (evidence-based findings), **what is prevented** (boundary enforcement) and **whether the fix worked** (regression and $pass^k$). The six improvements in Section 5 close the main design gaps: honest provider support, layered evaluation, explicit replay limits, real enforcement boundaries, domain-specific metrics and phased scope.

With Phase 0 completed, the design becomes implementation-ready. Building in the stated order produces a working, demonstrable workflow after every phase, and the held-out benchmark ensures that the platform's own claims are measured rather than assumed.

Note on figures

Durations, thresholds and targets in this report are planning estimates. Results must be reported only from actual measurements.